

Multi Backup

User Manual

Secure • Incremental • Cross-platform

RELEASE CANDIDATE 1 | EN | NOVEMBER 2026

Contents

01 The workspace and first launch

02 Installation and platform requirements

03 Dashboard tour

04 Choosing a source plugin

05 Preparing GitHub credentials

06 Preparing Dropbox credentials

07 Backup workflow and progress

08 CI artifact cleanup and safety

09 Resuming, restarting and migration

10 Understanding encrypted archive layout

11 Browsing and previewing content

12 Checkout and read-only mount

13 Create and protect encryption keys

14 Application settings

15 Update channels and verification

16 CLI recipes

17 Recovery drills and operational checklist

18 Security limits and troubleshooting

19 Release channel and native support matrix

20 Roadmap and reporting issues

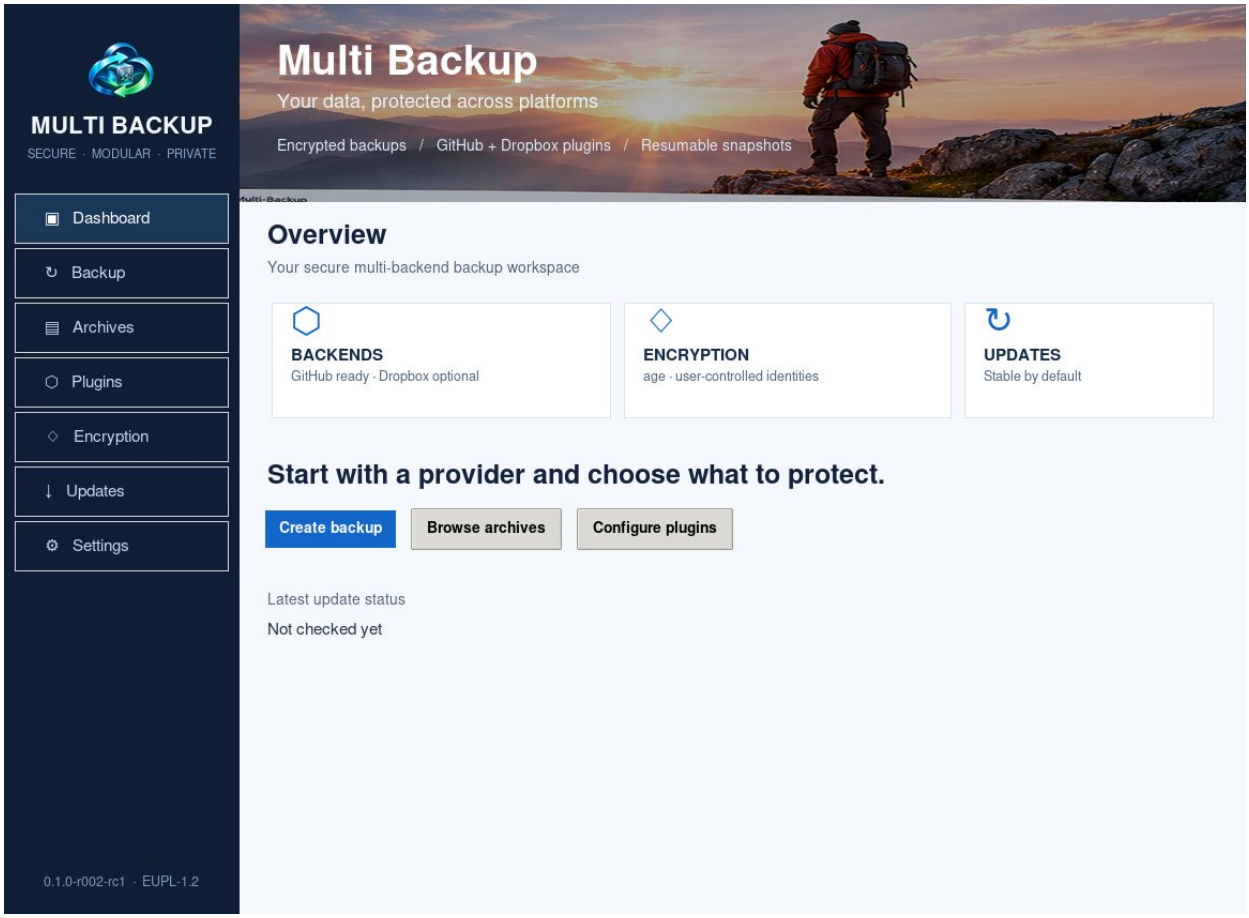
Note: screens are real captures of the RC Linux GUI, not fictitious live backups.

01 The workspace and first launch

Multi Backup combines independent source plugins under one encrypted backup workspace. The first release candidate preserves the GitHub disaster recovery engine and adds a Dropbox read-only incremental backup plugin.

The navigation sidebar provides Dashboard, Backup, Archives, Plugins, Encryption, Updates and Settings. Each screen works with the same application configuration and underlying CLI.

- Review the enabled plugins before starting a job.
- Set a private output directory on an encrypted volume.
- Keep your age private key separate from backup media.



Actual running dashboard captured on Linux. Other operating systems may render controls differently.

02 Installation and platform requirements

Install from the release page matching your platform: DMG on macOS 13+ (Apple Silicon or Intel), EXE/ZIP on Windows 11 x64, or RPM for Rocky/AlmaLinux 8, 9 and 10. Until signatures are configured, these are RC evaluation packages.

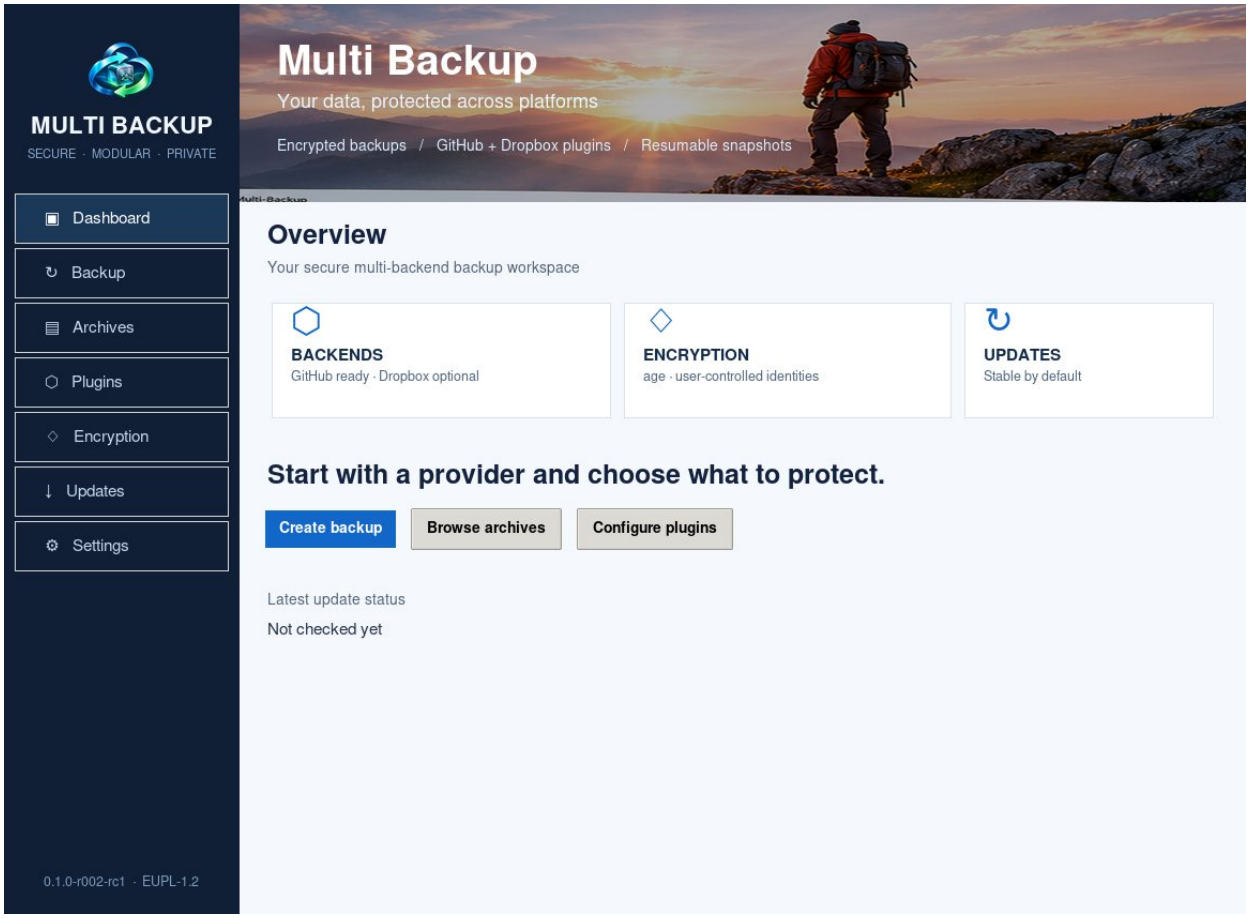
For source use, install Python 3.11+, dependencies from requirements.txt and the external age, git, Git LFS and GitHub CLI programs. The Dropbox engine requires age and a token with minimum read permissions.

- Confirm the installer signature and SHA-256 digest before proceeding.
- Allow the OS to install the CLI helper tools if they were not bundled.
- Use gh auth login for GitHub; keep Dropbox tokens out of shell history.

03 Dashboard tour

Dashboard is designed for quick orientation. Three cards show the available providers, the encryption model and the update channel. Use Create backup to open the job configuration or Browse archives to inspect finished snapshots.

The application is deliberately not a cloud synchronization service: source data is never overwritten by restore actions without a user-selected destination.



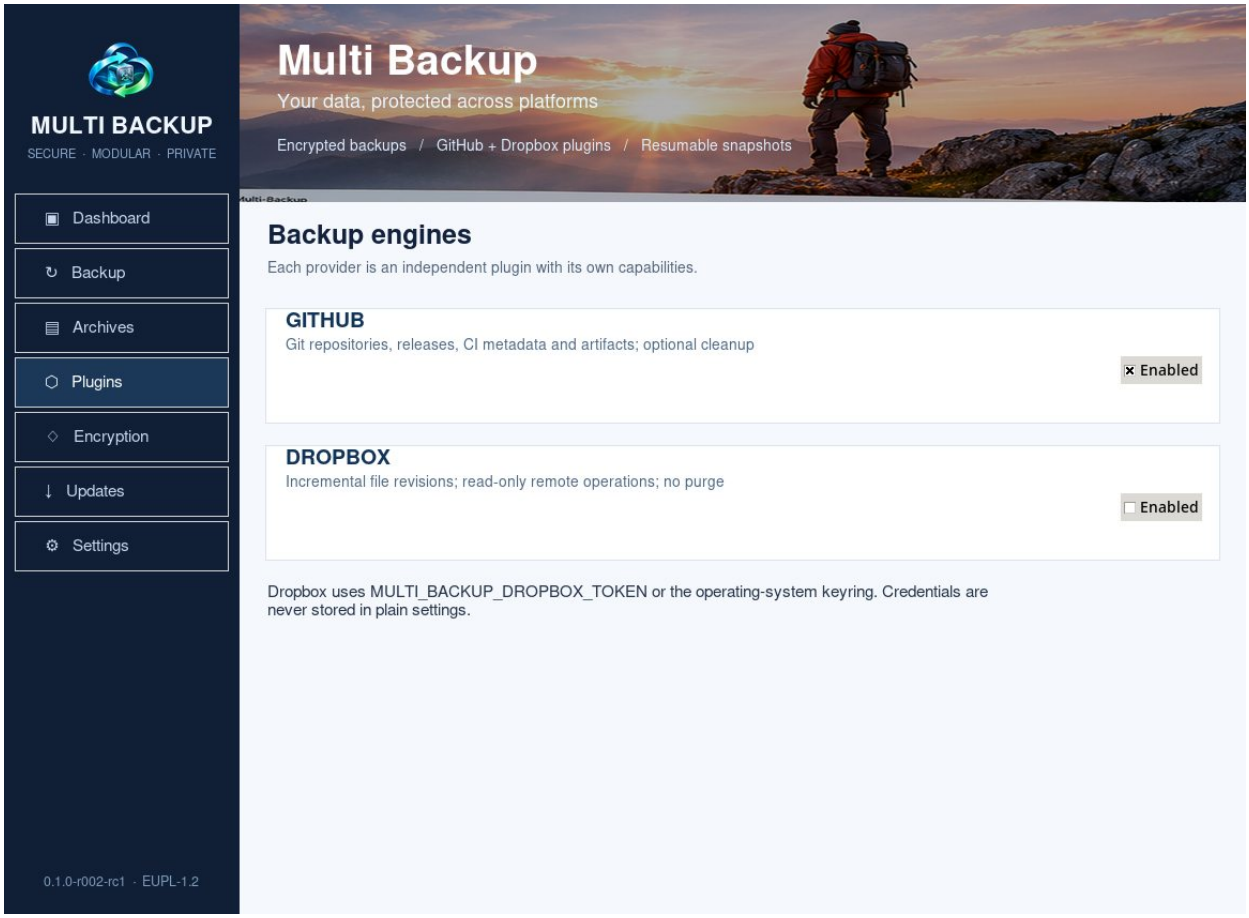
Dashboard landing page with photographic header and shortcuts.

04 Choosing a source plugin

Open Plugins to enable the backend required for a job. GitHub is enabled by default; Dropbox is disabled until the user explicitly switches it on. The backend selector in Backup determines which source engine runs.

External plugins are not imported automatically from arbitrary local paths. The current plugin registry exposes only the packaged, reviewed backends.

- GitHub supports repository metadata, release assets, CI logs and optional artifact cleanup.
- Dropbox supports read-only file revisions and incremental encrypted storage.
- Dropbox never calls remote delete or purge APIs.



Plugin settings page. Change the enabled state before switching backend.

05 Preparing GitHub credentials

Authenticate the GitHub CLI with gh auth login and grant access to the repositories you want to preserve. GitHub returns metadata, but NOT the original values of existing Secrets.

Use an external secure directory if you intentionally want to include known original secret values in an encrypted disaster recovery archive. Exports from CI and release registries may have independent retention rules.

- Never paste tokens or credentials into README files or issue trackers.
- Dry-run CI artifact cleanup before performing any deletion.
- Verify that release publishing artifacts remain protected.

06 Preparing Dropbox credentials

Create a Dropbox API access token with read-only permissions. Enable Dropbox in Plugins and supply the token using MULTI_BACKUP_DROPBOX_TOKEN or an OS credential manager entry under service multi-backup and user dropbox.

The Dropbox engine reads folder listings and file downloads, calculates a stable key from file ID and revision, and reuses existing encrypted files when their SHA-256 checksum matches. It writes a separate encrypted manifest for each cycle.

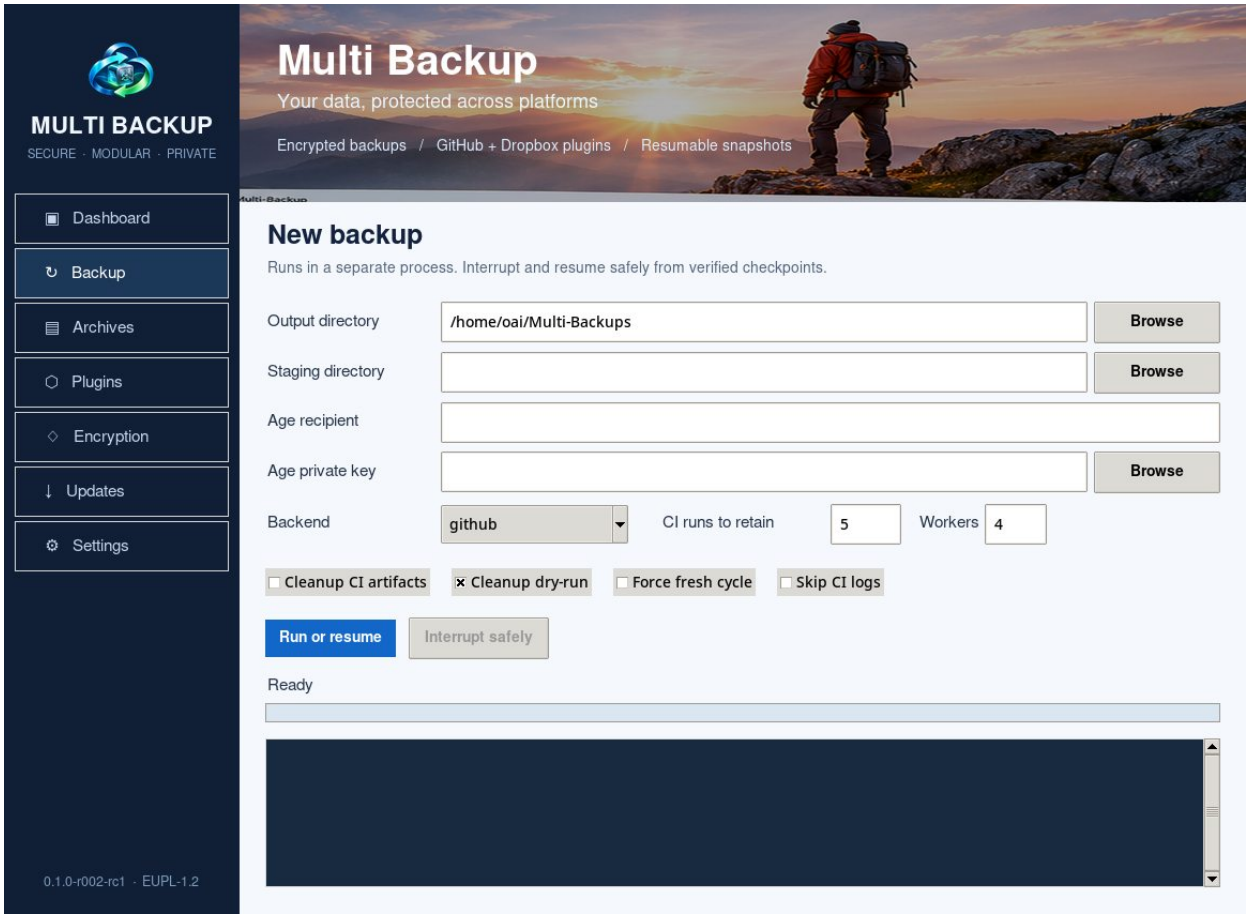
- No remote delete or modify calls are used.
- Local old revisions are never purged.
- Sharing ACLs and team admin settings are not yet exported in this RC.

07 Backup workflow and progress

In Backup, select the backend, output and staging locations, and an age recipient. The output directory must be private; stage files on an encrypted volume. Press Run or resume to start a child process.

The progress bar reflects the job state emitted by the CLI, while the log lists operations, warnings, and cleanup summaries. Interrupt safely leaves previous atomic archives and checkpoint journals intact.

- Do not use a shared or unencrypted staging volume.
- For GitHub, repeat the exact command to resume from the last verified shard.
- Select Force fresh cycle when you intentionally need a completely new run.



Backup page with source selection, encryption keys, cleanup simulation and progress bar.

08 CI artifact cleanup and safety

The optional GitHub artifact cleanup runs BEFORE the GitHub backup. It keeps the configured number of main-branch CI executions (default five), respects tag-associated runs, and dynamically protects artifacts needed for distribution, or whose dependencies are uncertain.

Use Cleanup dry-run first. Real deletion is irreversible. As work proceeds, the CLI shows per-artifact updates, per-repository summaries, and an overall report. Cleanup does not delete workflow histories, releases or Git repository references.

- Explicitly opt in to cleanup; it is off by default.
- Review dynamic protection decisions; unknown dependencies stay preserved.
- Do not assume an external software repository is independent from an old artifact without checking.

CLI example / Esempio CLI

```
multi-backup backup --backend github --cleanup-ci-artifacts --cleanup-ci-dry-run --keep-ci-runs 5
```

09 Resuming, restarting and migration

The GitHub engine saves an integrity-checked checkpoint for each completed metadata and repository archive. The current cycle is automatically resumed when the same configuration is used after interruption.

The --restart option produces a distinct backup cycle without overwriting the earlier completed archive. Older monolithic encrypted metadata archives are recognized and can be repartitioned with the correct age identity and recipient; originals remain immutable until new shards are verified.

- Do not remove .state until a full backup and restore validation have completed.
- An interrupted download may need to restart its current file; prior completed shards are retained.

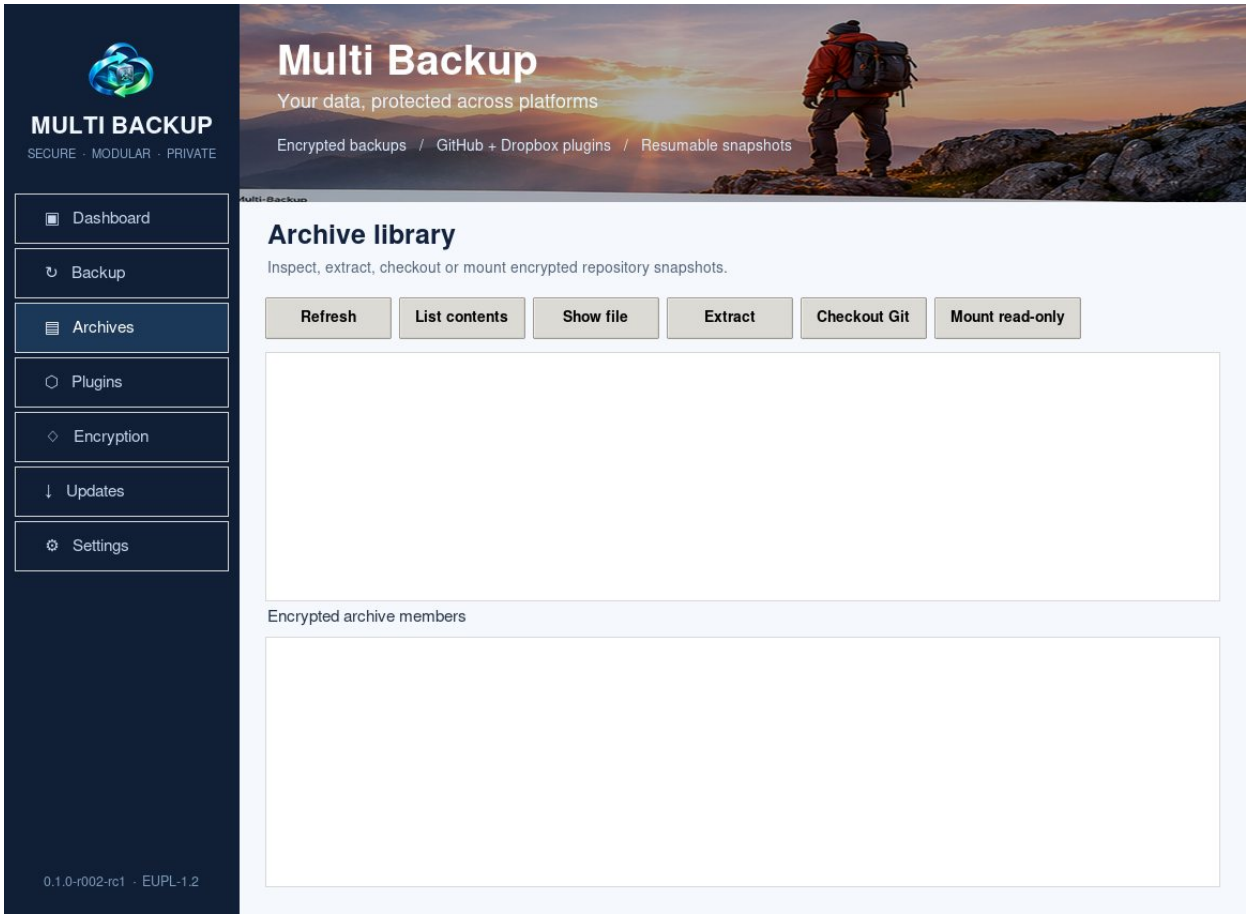
CLI example / Esempio CLI

```
multi-backup backup --backend github --recipient age1EXAMPLE --restart --progress
```

10 Understanding encrypted archive layout

The GitHub layout separates account/common metadata, per-repository settings, issues, releases and individual CI run archives; Git mirrors are kept in distinct encrypted archives, so unchanged snapshots may be deduplicated.

The Dropbox layout contains encrypted objects and versioned encrypted manifests. Each file revision can be reused across backup cycles; there is no remote or local purge. Sidecar SHA-256 digests detect accidental corruption.



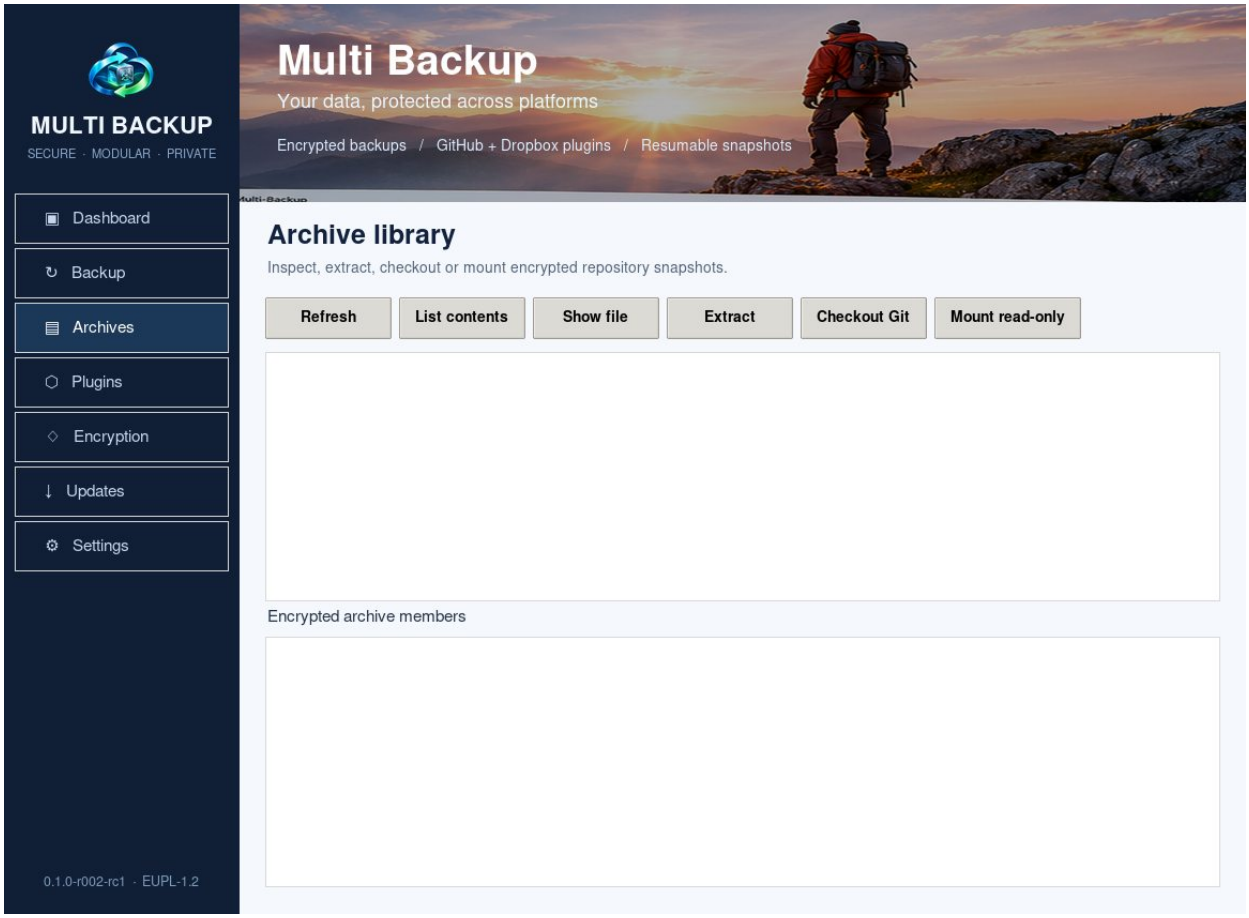
Archive explorer showing available encrypted metadata and repository archives.

11 Browsing and previewing content

Select an archive in the Archives page and provide the corresponding age private key. List contents to inspect member names. Show file displays a text or JSON member without extracting the entire archive into a user directory.

The CLI offers list, browse, view, show and extract with equivalent functions. Large archives are streamed through the decryption layer, and paths are validated before any extraction.

- Do not extract archives onto shared drives.
- Choose a new destination directory and review permissions.
- Git mirrors must be checked out to a fresh path.



Archive navigation and file inspection actions.

12 Checkout and read-only mount

Checkout Git converts a saved bare mirror into a working tree at a new destination. Mount read-only uses bindfs on Linux/macOS, or subst on Windows after preparation of a temporary decrypted copy.

The temporary mount workspace contains readable files; keep it on encrypted storage and unmount after use. The tool does not provide an always-encrypted virtual filesystem without disk staging.

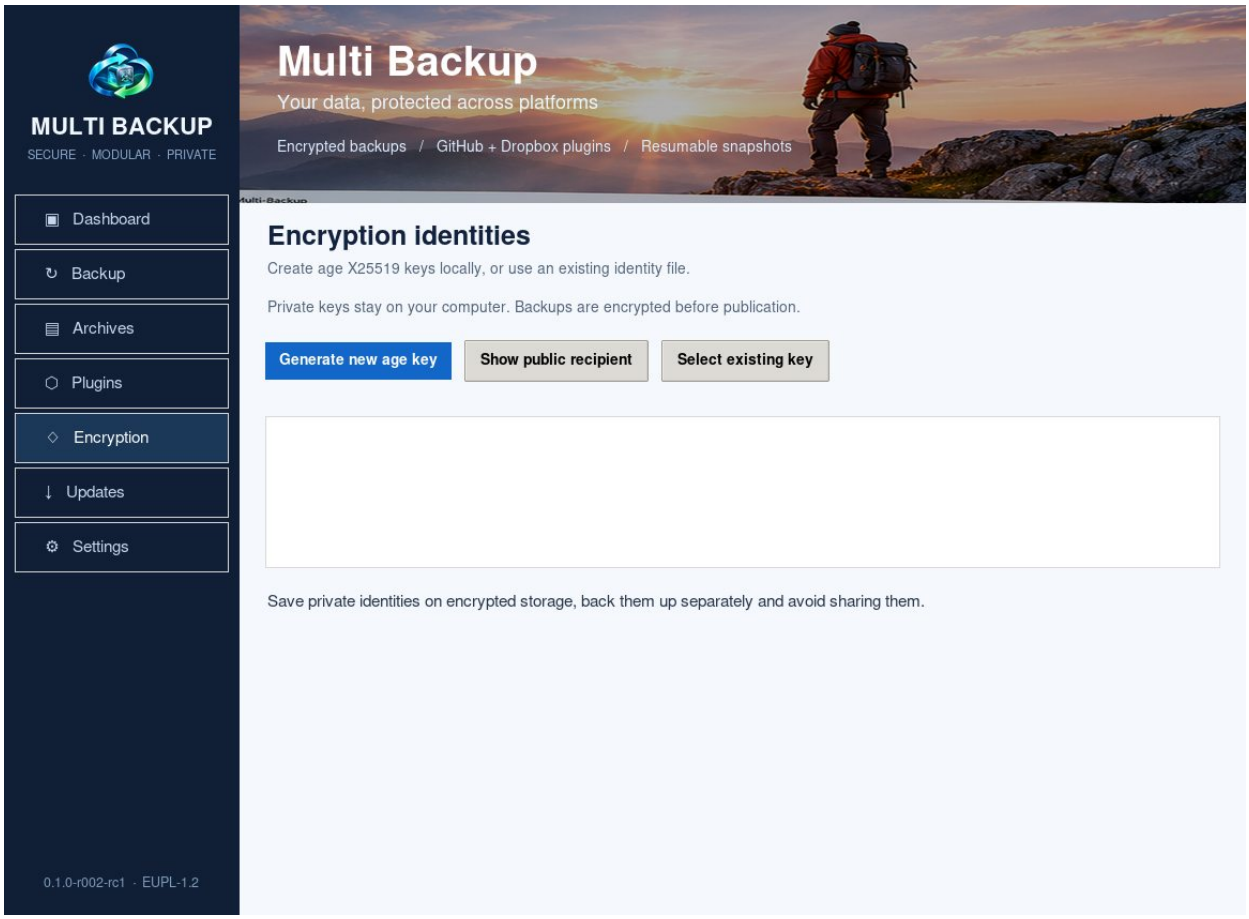
- Install and configure bindfs where supported.
- Use unmount after inspection to remove the private temporary extraction.

13 Create and protect encryption keys

Open Encryption and generate a new age X25519 identity. The CLI keys generate subcommand creates an identity in a private application configuration folder with owner-only permissions and prints only the public recipient.

The recipient string may be stored in application settings. The identity file must not be bundled with your ciphertext backups: keep separate encrypted/offline recovery copies and test the decryption process regularly.

- A lost age private key means the encrypted data is not recoverable.
- Never share private key contents in screenshots or support logs.
- Some platform key management requires installation of age-keygen.



Real key-management screen. Private key values are never displayed.

14 Application settings

Settings stores non-secret preferences such as backup location, enabled plugins, retention and update channel. On macOS settings are stored under the user Application Support folder; on Windows under APPDATA; on Linux under XDG_CONFIG_HOME or ~/.config.

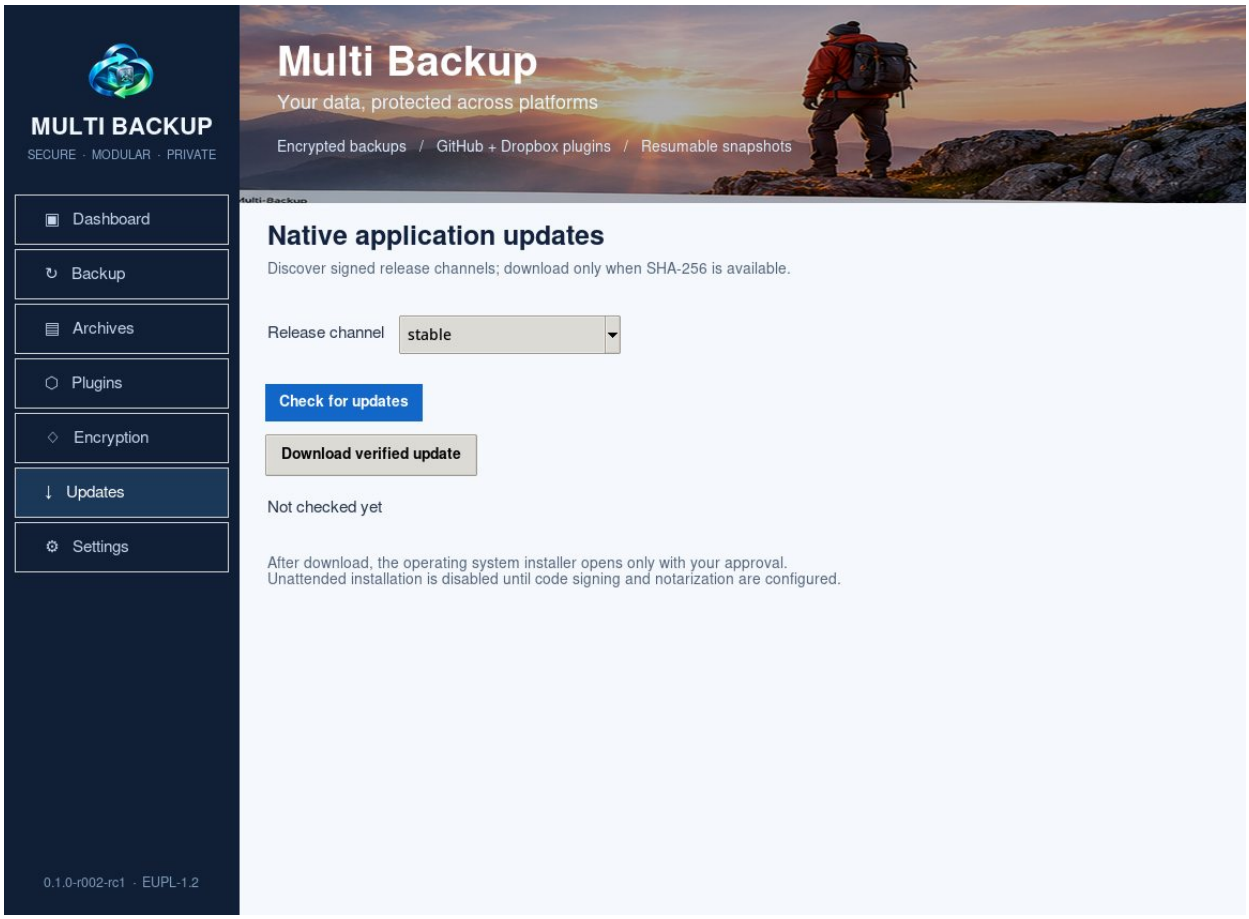
The settings file is kept private where supported. Dropbox access tokens are excluded from JSON settings and must live in the environment or protected OS keyring.

15 Update channels and verification

Stable is the default channel. Early release is selectable in Settings or the Updates screen. Early builds are uploaded after a [full-ci] commit on main, while stable releases require a version tag and a passing native build matrix.

The RC checks for updates at launch and periodically (default six hours). It can stage an update only when a GitHub-provided SHA-256 digest is present; it never silently launches unverified installers. OS signing is required for a production-grade unattended updater.

- Use stable unless you are testing RC builds.
- Validate publisher signatures before installing.
- Rolling early release packages may change without backward-compatibility guarantees.



Updates page with release channel selection.

16 CLI recipes

The same actions are available by CLI. Select backend on each backup invocation; plugins can be enabled or disabled from the CLI. Use --progress for a progress bar and --json-progress to integrate with scripts.

Run --help for each subcommand. Example commands below cover creation, key management, retention simulation and channel selection.

CLI example / Esempio CLI

```
multi-backup --version
multi-backup plugins
multi-backup plugin enable dropbox
multi-backup keys generate --name primary
multi-backup backup --backend github --progress
multi-backup settings channel early-release
multi-backup updates check
```

17 Recovery drills and operational checklist

A backup is valuable only after recovery can be demonstrated. Regularly list the archive catalog, verify the SHA-256 sidecars, decrypt a small metadata file, checkout one Git mirror and confirm its Git history.

For Dropbox, restore a selected revision into a safe local directory (never over live source data), and compare its content to the expected revision. Document the identities and the incident-response procedure outside the archived storage.

- Keep encryption identities offline and separate.
- Test restore on a clean host before relying on the backup.
- Review incomplete counts and CI cleanup reports.

18 Security limits and troubleshooting

If a backup fails, inspect the CLI status and checkpoint before rerunning. Missing gh credentials, unavailable age, insufficient private staging permissions or expired API tokens are frequent causes. The resume mechanism avoids overwriting valid completed archives.

Remote backup metadata does not include original GitHub Secrets values or every external system setting. Native packages are release-candidate artifacts until full platform integration, signing and security review complete.

- Error 401/403: check credentials and scope.
- Invalid archive: verify checksum, identity and archive path.
- Old metadata: supply original age identity plus new recipient for migration.

19 Release channel and native support matrix

Multi Backup targets macOS Ventura 13 or later on Intel and Apple Silicon, Windows 11 x64, and Linux RPM distributions RockyLinux/AlmaLinux 8, 9 and 10. Build jobs use target OS runners or containers. All jobs must pass for release publication.

Early release assets are published under the rolling early-release tag on GitHub. Stable uses a version tag such as v0.1.0. RPM repository definitions remain disabled until signing and trust distribution are configured.

20 Roadmap and reporting issues

The first release candidate is a test milestone, not a final production certification. The next milestones are native installer launch tests, encrypted legacy migration test cases, live Dropbox integration and end-to-end install/update/restore validation.

Report defects to the issue tracker of desalvo/multi-backup with version, platform, exact CLI parameters (never secrets), sanitized logs and a brief reproduction. The handoff in docs/HANDOFF.md supports continuation of development.